

Entity Relationship Diagram

mermaid

erDiagram

USER ||--o{ WALLET : "has (1:N)"

WALLET ||--o{ TRANSACTION : "records (1:N)"

USER {

string id PK "UUID"

string email UK "Unique"

string name

string password "Hashed"

datetime createdAt

datetime updatedAt

}

WALLET {

string id PK "UUID"

string userId FK "References User.id"

float balance "Default: 0.0"

string currency "Default: USD"

WalletStatus status "ACTIVE | INACTIVE | FROZEN"

int version "Optimistic Locking"

datetime createdAt

datetime updatedAt

}

TRANSACTION {

string id PK "UUID"

string walletId FK "References Wallet.id"

float amount

TransactionType type "CREDIT | DEBIT"

TransactionCategory category "DEPOSIT | TRANSFER_IN | TRANSFER_OUT | WITHDRAWAL"

TransactionStatus status "PENDING | SUCCESS | FAILED"

float balanceBefore

```

float balanceAfter

string referenceId "Shared across transfer pair"

string senderWalletId "Optional"

string receiverWalletId "Optional"

string description "Optional"

string metadata "Optional (JSON)"

datetime createdAt
}

IDEMPOTENCY_RECORD {

string id PK "UUID"

string key UK "Idempotency-Key Header"

string userId "Optional"

string requestPath

string requestHash "Optional"

int statusCode

string responseBody

datetime expiresAt

datetime createdAt

}

```

Enums

Enum	Allowed Values	Description
WalletStatus	ACTIVE, INACTIVE, FROZEN	Lifecycle status of a wallet
TransactionType	CREDIT, DEBIT	Accounting direction
TransactionCategory	DEPOSIT, TRANSFER_IN, TRANSFER_OUT, WITHDRAWAL	Business intent of transaction
TransactionStatus	PENDING, SUCCESS, FAILED	State of the transaction

Models & Constraints Breakdown

1. users Table

- **id**: String (UUID) — Primary Key
- **email**: String — Unique index
- **name**: String
- **password**: String (bcrypt hashed)
- **createdAt / updatedAt**: DateTime
- **Relations**: wallets (1-to-many with cascade delete)

2. wallets Table

- **id**: String (UUID) — Primary Key
- **userId**: String — Foreign Key $\rightarrow$ User.id (onDelete: Cascade)
- **balance**: Float — Default 0.0
- **currency**: String — Default "USD"
- **status**: WalletStatus — Default ACTIVE
- **version**: Int — Default 0 (used for optimistic locking)
- **Constraints & Indexes**:
 - @@unique([userId, currency]): Enforces **one wallet per currency per user**
 - @@index([userId]): Fast user lookup

3. transactions Table

- **id**: String (UUID) — Primary Key
- **walletId**: String — Foreign Key $\rightarrow$ Wallet.id (onDelete: Cascade)
- **amount**: Float
- **type**: TransactionType (CREDIT or DEBIT)
- **category**: TransactionCategory (DEPOSIT, TRANSFER_IN, TRANSFER_OUT, WITHDRAWAL)
- **status**: TransactionStatus — Default SUCCESS
- **balanceBefore**: Float — Snapshot before transaction
- **balanceAfter**: Float — Snapshot after transaction
- **referenceId**: String? — Unique ID linking matching sender debit & receiver credit transactions
- **senderWalletId / receiverWalletId**: String?
- **description / metadata**: String?
- **createdAt**: DateTime — Default now()

- **Indexes:**
 - `@@index([walletId, createdAt]):` Efficient chronological history queries
 - `@@index([walletId, type]):` Fast filtering by CREDIT / DEBIT
 - `@@index([referenceId]):` Fast lookup for linked transfer pairs

4. idempotency_records Table

- **id:** String (UUID) — Primary Key
- **key:** String — Unique Idempotency-Key
- **userId:** String?
- **requestPath:** String
- **requestHash:** String?
- **statusCode:** Int — Cached HTTP response code
- **responseBody:** String — Cached JSON response
- **expiresAt:** DateTime — TTL expiration
- **createdAt:** DateTime — Default now()
- **Indexes:**
 - `@@index([key]), @@index([expiresAt])`

Prisma Schema File (

```
prisma/schema.prisma)

prisma
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum WalletStatus {
  ACTIVE
```

```
    INACTIVE
    FROZEN
}
enum TransactionType {
    CREDIT
    DEBIT
}
enum TransactionCategory {
    DEPOSIT
    TRANSFER_IN
    TRANSFER_OUT
    WITHDRAWAL
}
enum TransactionStatus {
    PENDING
    SUCCESS
    FAILED
}
model User {
    id    String  @id @default(uuid())
    email String  @unique
    name  String
    password String
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    wallets Wallet[]
    @@map("users")
}
model Wallet {
    id      String  @id @default(uuid())
    userId  String
```

```

user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
balance   Float      @default(0.0)
currency  String      @default("USD")
status    WalletStatus @default(ACTIVE)
version   Int         @default(0)
createdAt DateTime    @default(now())
updatedAt DateTime    @updatedAt
transactions Transaction[]

@@unique([userId, currency], name: "user_currency_unique")
@@index([userId])
@@map("wallets")
}

model Transaction {
  id          String      @id @default(uuid())
  walletId    String
  wallet      Wallet      @relation(fields: [walletId], references: [id], onDelete: Cascade)
  amount      Float
  type        TransactionType
  category    TransactionCategory @default(DEPOSIT)
  status      TransactionStatus @default(SUCCESS)
  balanceBefore Float
  balanceAfter Float
  referenceId String?
  senderWalletId String?
  receiverWalletId String?
  description String?
  metadata    String?
  createdAt   DateTime      @default(now())
  @@index([walletId, createdAt])
  @@index([walletId, type])
  @@index([referenceId])

```

```
    @@map("transactions")
}

model IdempotencyRecord {
    id      String @id @default(uuid())
    key     String @unique
    userId  String?
    requestPath String
    requestHash String?
    statusCode Int
    responseBody String
    expiresAt DateTime
    createdAt DateTime @default(now())
    @@index([key])
    @@index([expiresAt])
    @@map("idempotency_records")
}
```